

TRYHACKME - RELATÓRIO TÉCNICO

Análise de Segurança Web no Front-end

HTML, JavaScript, Exposição de Dados Sensíveis e Injeção de HTML

Autor: Amilcar Junior

Plataforma: TryHackMe

Data: 27 de agosto de 2026

Sumário

1. Arquitetura web e modelo cliente-servidor	3
2. Estrutura de uma página HTML	4
3. Manipulação prática de conteúdo HTML	5
4. JavaScript e manipulação do DOM	6
5. Exposição de dados sensíveis no front-end	7
6. Análise prática do código-fonte	8
7. Injeção de HTML	9
8. Exploração controlada	10
9. Controles de segurança recomendados	11
10. Conclusão	12

1. Arquitetura web e modelo cliente-servidor

Este relatório documenta um laboratório autorizado sobre o funcionamento de aplicações web e os riscos de segurança presentes no front-end. Quando um usuário acessa um site, o navegador envia uma requisição ao servidor, que processa a solicitação e devolve os recursos necessários para renderizar a página.



Figura 1: Fluxo de requisição e resposta entre navegador, internet e servidor web.

Front-end: código executado ou interpretado pelo navegador, incluindo HTML, CSS e JavaScript.

Back-end: serviços executados no servidor, responsáveis por regras de negócio, autenticação, acesso a dados e geração de respostas.

Relevância para Segurança da Informação

O navegador recebe e processa o código do front-end. Por isso, segredos enviados ao cliente não podem ser considerados protegidos apenas por estarem escondidos visualmente. Tudo que chega ao navegador pode ser inspecionado pelo usuário.

O laboratório foi executado exclusivamente no ambiente educacional do TryHackMe. As técnicas registradas são apresentadas para estudo, identificação de falhas e desenvolvimento de controles defensivos.

2. Estrutura de uma página HTML

HTML é a linguagem de marcação usada para organizar a estrutura e o conteúdo de uma página. Os elementos são definidos por tags, enquanto atributos fornecem informações adicionais, como classes, identificadores, endereços de imagens e destinos de links.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page Title</title>
  </head>
  <body>
    <h1>Example Heading</h1>
    <p>Example paragraph..</p>
  </body>
</html>
```

Figura 2: Estrutura básica de um documento HTML.

Elemento	Finalidade	Exemplo
DOCTYPE	Informa ao navegador o padrão do documento.	<!DOCTYPE html>
html	Elemento raiz que contém a página.	<html>...</html>
head	Armazena metadados e o título da página.	<title>...</title>
body	Contém o conteúdo exibido ao usuário.	<h1>, <p>,
id	Identifica um elemento de forma única.	id="demo"
class	Agrupar elementos para estilo ou comportamento.	class="login"

O código HTML pode ser consultado por meio da opção de visualizar o código-fonte ou pelas ferramentas de desenvolvedor. Essa transparência é esperada: o erro de segurança ocorre quando o desenvolvedor inclui no front-end informações que deveriam permanecer no servidor.

3. Manipulação prática de conteúdo HTML

Na primeira atividade prática, foi utilizado o editor HTML do laboratório para analisar a estrutura de uma página de imagens. O comentário existente no código indicava o local em que uma nova imagem deveria ser adicionada.

HTML Code

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>TryHackMe HTML Editor</title>
5   </head>
6   <body>
7     <h1>Cat Website!</h1>
8     <p>See images of all my cats!</p>
9     <img src='img/cat-1.jpg'>
10    <img src='img/cat-2.'>
11    <!-- Add dog image here -->
12  </body>
13 </html>
```

Render HTML Code

Figura 3: Código original com indicação para inclusão de uma nova imagem.

```
<img src='img/dog-1.png'>
```

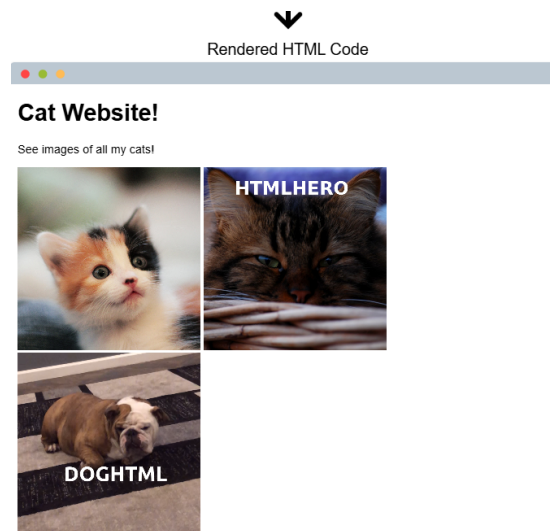


Figura 4: Página renderizada após a inclusão do novo elemento HTML.

Resultado

A alteração confirmou a relação direta entre o código HTML e o conteúdo renderizado. A atividade também reforçou a importância de revisar comentários, caminhos e dados deixados no código entregue ao navegador.

4. JavaScript e manipulação do DOM

JavaScript adiciona comportamento e interatividade às páginas. No exercício, o método `document.getElementById` localizou o elemento com o identificador `demo`, e a propriedade `innerHTML` substituiu seu conteúdo.

```
document.getElementById("demo").innerHTML = "Hack the Planet";
```

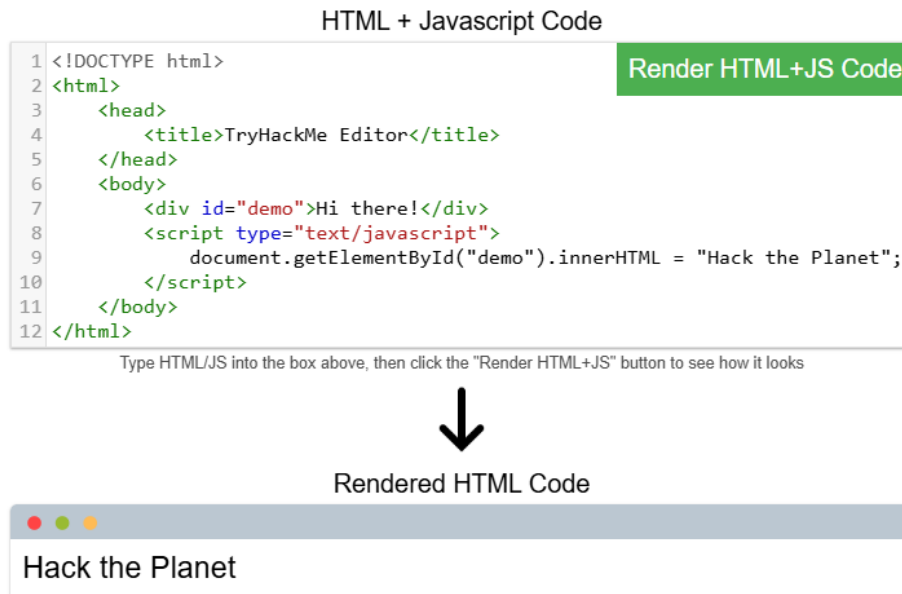


Figura 5: Alteração dinâmica do conteúdo da página por JavaScript.

Ponto de segurança

`innerHTML` interpreta marcação HTML. Quando recebe diretamente dados controlados pelo usuário, pode permitir injeção de conteúdo. Para exibir texto, `textContent` é normalmente mais seguro porque não interpreta tags HTML.

5. Exposição de dados sensíveis no front-end

A exposição de dados sensíveis ocorre quando informações que deveriam estar protegidas são enviadas ao cliente em texto legível. Comentários HTML, arquivos JavaScript, campos ocultos e respostas de API podem revelar credenciais de teste, rotas internas, identificadores, tokens ou detalhes de implementação.

```
<!DOCTYPE html>
<html>
  <head>
    <title>Fake Website</title>
  </head>
  <body>
    <form>
      <input type='text' name='username'>
      <input type='password' name='password'>
      <button>Login</button>
      <!-- TODO: remove test credent [CREDENCIAL OCULTADA] -->
    </form>
  </body>
</html>
```

Figura 6: Exemplo didático de credenciais deixadas em um comentário HTML.

Exposição	Possível impacto	Controle principal
Credenciais	Acesso não autorizado a contas ou sistemas.	Nunca armazenar segredos no cliente.
Rotas ocultas	Descoberta de painéis e funções administrativas.	Aplicar autorização no servidor.
Tokens e chaves	Uso indevido de APIs e sessões.	Manter segredos no back-end e rotacionar chaves.
Comentários	Divulgação de dados de teste ou decisões internas.	Revisar e remover antes da publicação.

Visão defensiva

Ocultar um elemento na interface não protege o dado. Autenticação e autorização devem ser validadas no servidor, independentemente do que o front-end mostra ao usuário.

6. Análise prática do código-fonte

No site vulnerável do laboratório, as ferramentas de desenvolvedor foram utilizadas para inspecionar os elementos da página de login. A análise revelou um comentário esquecido no código-fonte contendo credenciais de teste.

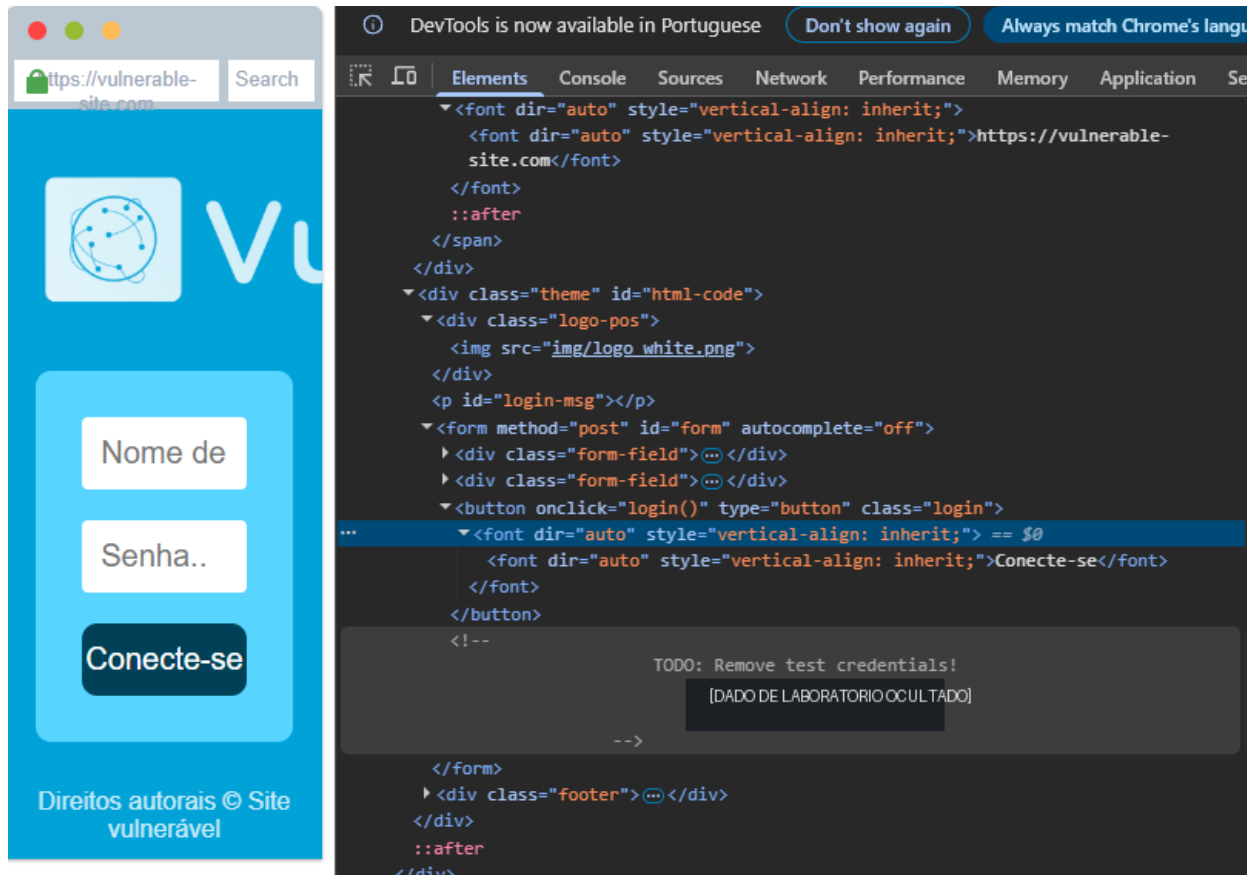


Figura 7: Inspeção do HTML com as credenciais específicas do laboratório ocultas para publicação segura.

Procedimento executado:

1. Acesso à página de autenticação no ambiente autorizado.
2. Abertura das ferramentas de desenvolvedor e seleção da guia de elementos.
3. Inspeção do formulário e dos comentários HTML.
4. Identificação das credenciais de teste expostas e registro da evidência com os valores redigidos.

Impacto

Se credenciais válidas forem publicadas no front-end, qualquer visitante poderá recuperá-las. O controle correto é remover o segredo do código cliente, invalidar as credenciais expostas e revisar o processo de implantação.

7. Injeção de HTML

A injeção de HTML ocorre quando a aplicação recebe uma entrada controlada pelo usuário e a insere na página sem codificação ou sanitização adequada. O navegador interpreta a entrada como marcação, permitindo alterar a estrutura visual e criar elementos não previstos pela aplicação.

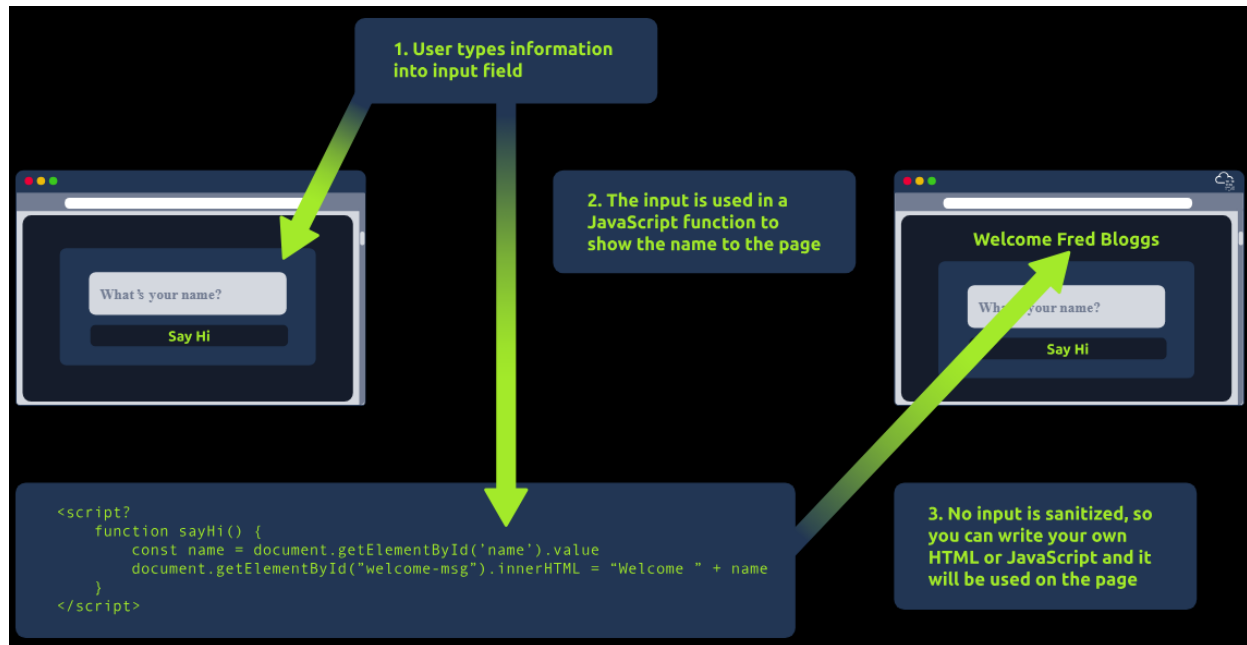


Figura 8: Fluxo em que a entrada do usuário é inserida na página por meio de innerHTML.

Risco	Exemplo de efeito
Alteração visual	Criação de títulos, mensagens ou conteúdo falso.
Links maliciosos	Redirecionamento do usuário para páginas externas.
Formulários falsos	Simulação de telas para captura de informações.
Engenharia social	Uso da confiança no domínio legítimo para enganar o usuário.

HTML Injection não é automaticamente XSS

O laboratório comprovou a interpretação de uma tag HTML injetada. Para classificar o resultado como Cross-Site Scripting seria necessário demonstrar a execução de JavaScript controlado pelo usuário. Manter essa distinção evita exagerar a conclusão técnica.

8. Exploração controlada

A página vulnerável solicitava um nome e exibia o valor informado em uma mensagem de boas-vindas. Como a aplicação utilizava innerHTML sem tratar a entrada, foi possível inserir uma tag de link no campo e observar sua renderização na página.

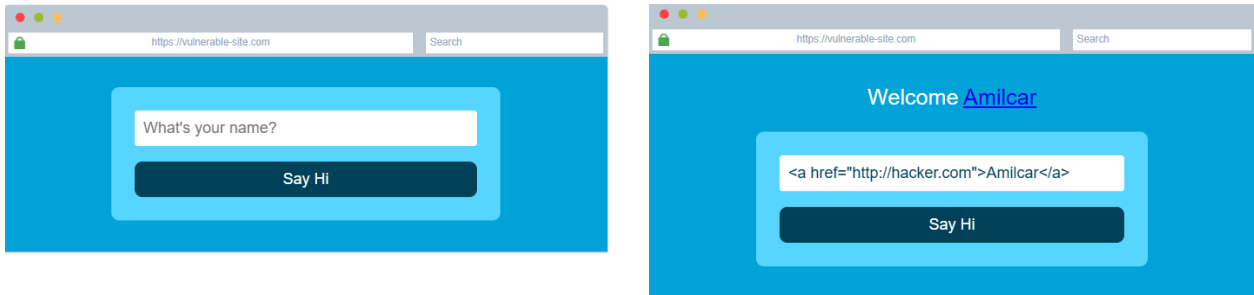


Figura 10: Link HTML interpretado pela página.

Figura 9: Campo original da aplicação.

Payload utilizado no ambiente de laboratório:

```
<a href="http://hacker.com">Amilcar</a>
```

Etapa	Evidência observada
Entrada	Uma tag HTML foi enviada pelo campo destinado ao nome.
Processamento	O JavaScript inseriu o valor diretamente em innerHTML.
Resultado	O navegador criou um link clicável dentro da mensagem.
Conclusão	A aplicação interpretou a entrada como HTML, confirmando a vulnerabilidade.

Escopo ético
O teste foi realizado somente no cenário vulnerável e autorizado do TryHackMe. Nenhuma aplicação pública ou sistema de terceiros foi testado.

9. Controles de segurança recomendados

A correção deve tratar a origem da vulnerabilidade, e não apenas bloquear um payload específico. Os principais controles identificados no laboratório são apresentados a seguir.

Controle	Aplicação prática	Risco reduzido
Codificação de saída	Converter caracteres especiais antes de exibir dados do usuário.	Interpretação de tags injetadas.
Uso de <code>textContent</code>	Exibir texto sem interpretar marcação HTML.	HTML Injection em conteúdo textual.
Sanitização	Permitir somente tags e atributos necessários quando HTML for realmente aceito.	Conteúdo ativo ou perigoso.
Segredos no servidor	Nunca incluir senhas, tokens ou chaves no front-end.	Exposição de dados sensíveis.
Autorização no back-end	Validar permissões para cada operação e recurso.	Acesso indevido por rotas descobertas.
Revisão e testes	Inspecionar código, comentários, dependências e artefatos antes da publicação.	Falhas introduzidas no desenvolvimento.
Content Security Policy	Restringir origens e tipos de conteúdo como camada adicional.	Impacto de injeções e carregamentos externos.
Monitoramento	Registrar entradas suspeitas e correlacionar eventos em WAF, proxy e aplicação.	Detecção tardia de abuso.

Exemplo de implementação mais segura

Ao exibir o nome do usuário, prefira: `element.textContent = 'Welcome ' + name;` O controle deve ser combinado com validação no servidor sempre que os dados também forem processados pelo back-end.

Para uma operação SOC ou MSS, payloads com tags HTML em parâmetros, formulários e logs podem funcionar como sinais de investigação. A análise deve correlacionar origem, frequência, rota afetada, resposta da aplicação e controles existentes.

10. Conclusão

O laboratório conectou o funcionamento básico de aplicações web a duas falhas de segurança observáveis na prática. Primeiro, a inspeção do código-fonte revelou credenciais de teste deixadas em um comentário HTML. Depois, uma entrada controlada foi interpretada como marcação por uma rotina JavaScript que utilizava innerHTML, confirmando uma vulnerabilidade de injeção de HTML.

A atividade demonstrou que informações presentes no front-end não podem ser tratadas como secretas e que toda entrada deve ser processada de acordo com o contexto em que será exibida. Também reforçou a diferença entre ocultar um elemento visualmente e realmente aplicar controles de autenticação e autorização no servidor.

Sob a ótica defensiva, as evidências podem apoiar revisão de código, resposta a incidentes, análise de logs, operação de WAF e investigação de comportamentos suspeitos em aplicações web. O resultado representa uma base prática para evoluir em Segurança de Aplicações, SOC, MSS e Blue Team.

Competências demonstradas

Arquitetura cliente-servidor; leitura de HTML; uso de ferramentas de desenvolvedor; manipulação básica do DOM; identificação de dados sensíveis no front-end; validação controlada de HTML Injection; análise de impacto; recomendação de controles; documentação técnica com evidências.

Achado	Evidência	Recomendação
Credenciais no código-fonte	Comentário HTML encontrado durante a inspeção.	Remover, invalidar e manter segredos no servidor.
HTML Injection	Tag de link renderizada a partir da entrada do usuário.	Codificar saída e evitar innerHTML para texto.

Declaração de uso responsável

Todos os testes descritos foram realizados em ambiente educacional autorizado. O documento não divulga credenciais específicas do exercício e não deve ser utilizado para testar sistemas sem permissão explícita.